Campus Event Planner

Mid-Project Progress

Xander Murphy & Joseph Gallucci

ASE 456 – Cross-Platform Development

Project Recap

Problem: students juggle classes, assignments, clubs and campus events across several apps and calendars.

Our app: one simple planner to keep it all in one place.

Major features

- Create, edit and delete events
- Categories: Class · Assignment · Club · Campus Event · Personal
- Upcoming list, calendar view, search & filter
- Mark events completed
- Saved on the device — no account or backend

Target platform: Android (Flutter, so iOS/web are possible later)

Progress Since the Proposal

Feature	Status
Bottom navigation: Home · Calendar · Events · Settings	Done
Create / edit / delete events	Done
Categories, location & description	Done
Mark events completed	Done
Search + category filter + "show completed"	Done
Calendar day view	*Partial* – no markers on days with events
Local storage (SharedPreferences)	Done
Unit / widget tests	**Pending** – default test still in repo
Reminders / notifications	**Pending** (planned "future" feature)

Compared to Our Milestones

Proposal phase	Planned	Where we are
1. Plan	Weeks 1–2	Done – PPP, repo, file structure
2. Build	Weeks 3–5	Done – screens, navigation, model
3. Features	Weeks 6–9	Done – CRUD, categories, search
4. Storage	Weeks 10–11	**Done early**
5. Test	Weeks 12–13	Next
6. Finalize	Weeks 14–15	Upcoming

On schedule, slightly ahead — storage was pulled forward because the app felt incomplete without saving.

Changes From the Proposal

- **Calendar:** used Flutter's built-in `CalendarDatePicker` instead of a calendar package
  → fewer dependencies, but no event markers yet
- **Simpler architecture first:** one shared event list instead of a full state-management layer
  → got a working app fast; we plan to refactor it toward our OOP/SOLID goals
- **Workflow:** so far most work went straight to `main` instead of branches + pull requests
  → switching to branches/PRs for the rest of the project

Live Demo

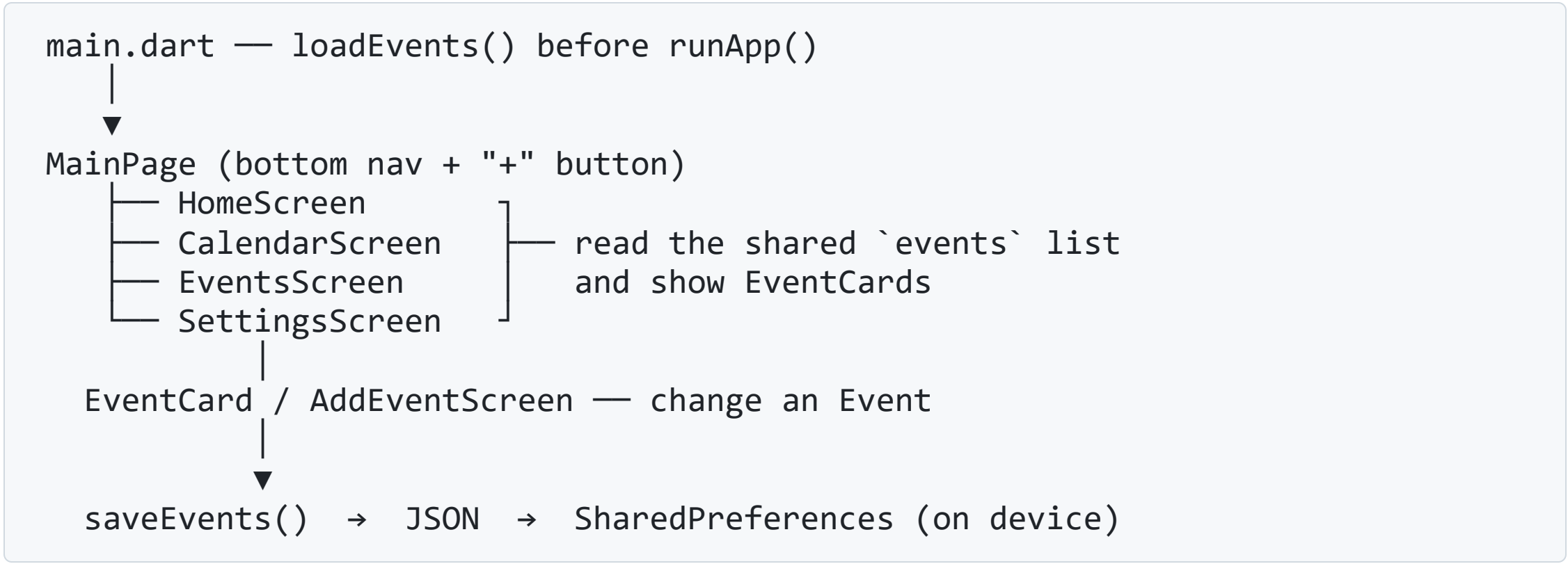
Android emulator

Technical Implementation

Area	What we use
Framework / language	Flutter, Dart
UI	Material widgets (`Scaffold` , `ListView` , `Card` , `BottomNavigationBar`)
Calendar	`CalendarDatePicker` (built into Flutter)
Date/time input	`showDatePicker` , `showTimePicker`
Storage	`shared_preferences` – events saved as JSON
Tools	VS Code / Android Studio, Android emulator, Git & GitHub

No backend, database server or external API — everything stays on the device.

How the Pieces Fit Together



`lib/` follows our proposal layout: **models · providers · services · screens · widgets**

Saving & Loading Events

```
// models/event.dart – each event turns itself into a Map
Map<String, dynamic> toMap() => {
  'title': title,
  'date': date.toIso8601String(),
  'category': category,
  'completed': completed, ...
};

// services/storage_service.dart
Future<void> saveEvents() async {
  final prefs = await SharedPreferences.getInstance();
  final list = events.map((e) => e.toMap()).toList();
  await prefs.setString('events', jsonEncode(list));
}
```

On startup `loadEvents()` does the reverse: read the string → `jsonDecode` → `Event.fromMap`.

Cross-Platform Approach

- **One Dart codebase** – Flutter draws the same UI on every platform
- Every package we use (`shared_preferences`, Material) supports Android, iOS, web and desktop
- Currently only the **`android/`** platform folder is in the repo
  → `flutter create --platforms=ios,web .` adds others without changing `lib/`
- **Screen sizes:** layouts are scrolling `ListView`s, so they adapt to phones, tablets and rotation
- Testing on the **Android emulator**; planning a physical-phone test

Concepts we learned

`StatefulWidget` + `setState` · async/await with `Future` · JSON serialization · passing callbacks between widgets · `Navigator.push`

Challenges So Far

Challenge	How we handled it
A nested Flutter project ended up inside the repo	Removed it and reset the file structure to match the PPP
Screens not refreshing after an event was added or edited	`await Navigator.push(...)` then `setState`; cards call an `onChanged` callback
Events disappeared when the app closed	Added `shared_preferences` + JSON saving
Event dates/times	Combined the date picker and time picker into one `DateTime`
Learning Flutter/Dart while building	Built the simplest working version first

Remaining Work

Must finish

- Replace the default counter test with real unit tests (model, search/filter, save/load) and widget tests
- Confirmation before "Delete" / "Delete all"
- Bug fixes and UI polish after testing on a physical Android device

Planned improvements

- Refactor toward our OOP/SOLID goals (event provider class, storage behind an interface)
- Calendar markers on days that have events
- Event reminders / notifications (stretch goal)

Timeline to Completion

When	Work
Weeks 12–13 (Test)	Unit + widget tests, delete confirmations, device testing, bug fixes
	Refactor event list into a provider class; calendar markers
Weeks 14–15 (Finalize)	UI polish, README & documentation, final presentation
If time allows	Reminders/notifications, iOS/web build

Team Contributions

Xander Murphy

- Repo setup and file structure from our PPP
- Screens & navigation: Home, Calendar, Events, Settings, add/edit form
- Event cards and category chips

Joseph Gallucci

- Created the event model
- Found the SharedPreferences package which allows data to survive app restarts

Remaining split (from our proposal)

- **Xander:** UI polish, calendar markers, delete confirmations, widget tests
- **Joseph:** provider/storage refactor, search/filter + model unit tests, reminders

Questions?

Campus Event Planner

github.com/Xander-Murphy/ASE456-CampusEvents-DuoProject

> Flutter • Dart • Android